

unified_contracts README

この README は、今回の要件を統合した **契約条件表現 + snapshot 管理 + representation 切替 + 外部ID連携** の最小モデルを説明します。

目的は次です。

- インディケーション後の約定データを表現する
- TARF / AKO Coupon Swap / 普通の Coupon Swap を表現する
- 同じ経済条件を `coupon_swap` と `option_bundle` の両方で扱えるようにする
- キャッシュフロー単位の個別修正も、`schedule` 全体の修正も、**その時点の契約全体のスナップショット** として履歴管理する
- 修正後も **元の圧縮契約情報** や **元の PeriodicSchedule** を保持する
- 外部システム連携時にこちらでIDを自動採番し、`trade` 単位・`option leg` 単位で記憶できるようにする

このため、設計は pricing engine や汎用 payoff compiler を目指さず、**canonical contract + trade representation + snapshot history + external linkage** に寄せています。

目次

1. 何を表現するか
2. 設計原則
3. パッケージ構成
4. ドメインモデル
5. 商品モデル
6. representation の考え方
7. スナップショット履歴の考え方
8. 外部システムID管理
9. 典型コード例
10. 今後の拡張ポイント

1. 何を表現するか

このパッケージは、主に次の契約を対象にしています。

- TARF
- AKO Coupon Swap
- 普通の Coupon Swap

また、同じ経済条件でも、業務上の取扱いとして次の 2 通りを持てます。

- `coupon_swap`
- `option_bundle`

たとえば TARF や AKOCS を、

- クーポンスワップとして扱う
- 通貨オプションの束として扱う

の両方をサポートします。

さらに、契約修正については

- `PeriodicSchedule` を変える
- 条件を変更する
- 個別の `cashflow` を修正する

のいずれであっても、**差分や `override` を持つのではなく、その時点の契約全体を新しいスナップショットとして積みます。**

ただし、元の `PeriodicScheduleSpec` や `EventSchedule` は `ScheduleArchive` として保持します。

2. 設計原則

2.1 商品ごとの `typed spec` を主役にする

今回は `ProductSpec(payoff, barrier, accrual, redemption, ...)` のような一般化は採らず、商品ごとの `dataclass` を主役にしています。

- `TARFSpec`
- `CouponSwapSpec`

- AK0CouponSwapSpec

これにより、承認対象として見たときに、商品ごとの意味が見えやすくなります。

2.2 共通化は本当に必要なところだけ

共通化しているのは次だけです。

- ProductIdentity
- Term
- ScheduleLike
- ObservationWindow
- TradeRepresentation
- CashflowRecord
- TradeSnapshot
- ExternalTradeRef
- ExternalComponentRef

2.3 representation を contract とは別軸で持つ

coupon_swap と option_bundle は product family ではなく、**取扱表現** として持ちます。

つまり、

- TARF という契約
- それを coupon_swap として扱う

を分離します。

2.4 修正は override ではなく snapshot として積む

ここが今回の設計の中心です。

- cashflow 単位の修正
- representation の切り替え
- schedule 全体の修正
- 契約条件の修正

のいずれでも、**承認対象となる契約全体の現在像** を TradeSnapshot として持ちます。

2.5 外部IDは contract 本体に埋め込まない

外部システム向けIDは契約の経済条件ではないため、TARFSpec や CouponSwapSpec には持たせません。

代わりに TradeDraft にぶら下げる形で、

- ExternalTradeRef
- ExternalComponentRef

として管理します。

3. パッケージ構成

```
unified_contracts/  
  common/  
    identity.py  
    terms.py  
    schedules.py  
    representations.py  
    cashflows.py  
    external_refs.py  
  
  products/  
    tarf.py  
    coupon_swap.py  
    __init__.py  
  
  workflow/  
    trade.py  
  
  services/  
    instantiate.py  
  
  outbound/  
    id_allocator.py  
  
examples.py  
README.md
```

4. ドメインモデル

大まかなモデルは次です。

Contract

- └─ TARFSpec
- └─ CouponSwapSpec
- └─ AK0CouponSwapSpec

TradeDraft

- └─ indication_snapshot
- └─ snapshots[]
- └─ external_trade_refs[]
- └─ external_component_refs[]
- └─ status

TradeSnapshot

- └─ contract
- └─ representation
- └─ schedule_archives
- └─ cashflows

Contract

契約の経済条件そのものです。

Representation

業務上どう扱うかです。

- CouponSwapRepresentation
- OptionBundleRepresentation

ScheduleArchive

元のスケジュール定義を保持します。

CashflowRecord

その revision 時点で有効だった cashflow 列です。

TradeSnapshot

revision ごとの契約全体像です。

ExternalTradeRef / ExternalComponentRef

外部システムへの連携時に割り当てた ID の記録です。

5. 商品モデル

5.1 TARFSpec

主な属性:

- underlying
- settlement_currency
- base_notional
- payoff_style
- payoff_schedule
- main_leg
- target
- final_fixing_treatment
- optional barrier fields

ポイント:

- TARF に固有の target を直接持つ
- redemption を別 component に切らない
- payoff schedule は PeriodicScheduleSpec または EventSchedule のまま保持できる

5.2 CouponSwapSpec

主な属性:

- underlying

- coupon_currency
- notional
- coupon_schedule
- coupon_formula
- pay_receive

5.3 AKOCouponSwapSpec

CouponSwapSpec に対して、さらに

- ako_level
- ako_window
- ako_condition
- action_on_breach

を持ちます。

6. representation の考え方

representation は、同じ契約を業務上どう扱うか を表します。

6.1 CouponSwapRepresentation

クーポンスワップとして取り扱うケースです。

この場合、1 イベントごとに

- coupon_leg

の cashflow を作ります。

6.2 OptionBundleRepresentation

通貨オプションの束として取り扱うケースです。

この場合、1 イベントごとに

- call_option_leg
- put_option_leg

などの cashflow を作ります。

重要なのは、これは **契約の経済条件そのものではなく、後続処理上の見せ方・扱い方** だということです。

7. スナップショット履歴の考え方

今回の中心は TradeSnapshot です。

```
TradeSnapshot(  
    revision_no=2,  
    created_at=...,  
    created_by="alice",  
    reason="manually adjusted first coupon",  
    contract=...,  
    representation=...,  
    schedule_archives=...,  
    cashflows=...,  
)
```

ポイントは次です。

7.1 修正の単位は「契約全体」

個別 cashflow 修正でも schedule 修正でも、結果として承認対象になるのは「その時点の契約全体」です。

なので、差分 object ではなく snapshot を積みます。

7.2 元の schedule は別に保持

schedule_archives には、元の圧縮 schedule を持ち続けます。

これにより、

- 現在の cashflow は手修正済み
- でも元はどんな PeriodicSchedule だったか分かる

を両立できます。

7.3 representation を変えたら cashflow もその view の現在像を持つ

coupon_swap から option_bundle に切り替える場合も、その revision 時点の representation と cashflows を丸ごと snapshot に入れます。

8. 外部システムID管理

外部連携では、次の 2 層をサポートします。

8.1 Trade 単位の外部ID

ExternalTradeRef で持ちます。

主な属性:

- system_name
- external_trade_id
- target_revision_no
- mode
- issued_at
- status

8.2 Option bundle の各 option leg 単位の外部ID

ExternalComponentRef で持ちます。

主な属性:

- system_name
- component_kind
- component_key
- external_component_id
- parent_external_trade_id
- target_revision_no

ここで component_key は、こちらのシステムで安定に識別する option leg key です。例:

- period_0_call

- period_0_put
- period_1_call
- period_1_put

OptionBundleRepresentation のときは、cashflow metadata にこの key を入れてあり、ExternalIdAllocator がそれを使って各 option leg にIDを振ります。

9. 典型コード例

9.1 初回生成

```
from datetime import datetime

from unified_contracts.common.representations import CouponSwapRepresentation
from unified_contracts.examples import make_tarf
from unified_contracts.services.instantiate import instantiate_trade_draft

trade = instantiate_trade_draft(
    draft_id="DRAFT-001",
    contract=make_tarf(),
    representation=CouponSwapRepresentation(),
    indication_payload={"quote_id": "Q-1001"},
    captured_at=datetime(2026, 4, 9, 9, 0, 0),
    created_by="alice",
)
```

9.2 個別 cashflow 修正

```
from datetime import datetime
from unified_contracts.common.cashflows import CashflowRecord

updated_cashflows = list(trade.cashflows)
first = updated_cashflows[0]
updated_cashflows[0] = CashflowRecord(
    cashflow_id=first.cashflow_id,
    source_schedule_id=first.source_schedule_id,
    source_event_index=first.source_event_index,
    view_kind=first.view_kind,
    leg_label=first.leg_label,
    currency=first.currency,
    amount_description="manually fixed coupon amount = 1,250,000 JPY",
    payment_date=first.payment_date,
    active=first.active,
    metadata={**first.metadata, "manual_edit": True},
)

trade.add_snapshot(
    created_by="alice",
    reason="manually adjusted first coupon after client negotiation",
    created_at=datetime(2026, 4, 9, 9, 30, 0),
    cashflows=updated_cashflows,
)
```

9.3 option bundle へ切り替え

```
from datetime import datetime
from unified_contracts.common.representations import OptionBundleRepresentation
from unified_contracts.services.instantiate import instantiate_trade_draft

option_view_cashflows = instantiate_trade_draft(
    draft_id="TEMP",
    contract=trade.contract,
    representation=OptionBundleRepresentation(),
    indication_payload={},
    captured_at=datetime(2026, 4, 9, 10, 0, 0),
).cashflows

trade.add_snapshot(
    created_by="bob",
    reason="re-book as option bundle view",
    created_at=datetime(2026, 4, 9, 10, 0, 0),
    representation=OptionBundleRepresentation(),
    cashflows=option_view_cashflows,
)
```

9.4 外部IDの採番

```
from datetime import datetime
from unified_contracts.outbound.id_allocator import ExternalIdAllocator

allocator = ExternalIdAllocator()
trade_ref = allocator.allocate_trade_ref(
    system_name="BOOKING_A",
    trade=trade,
    issued_at=datetime(2026, 4, 9, 10, 5, 0),
)

component_refs = allocator.allocate_component_refs_for_option_bundle(
    system_name="BOOKING_A",
    trade=trade,
    parent_external_trade_id=trade_ref.external_trade_id,
    issued_at=datetime(2026, 4, 9, 10, 5, 0),
)
```

10. 今後の拡張ポイント

10.1 snapshot 差分表示

モデルは snapshot を積むだけになっているので、UI での差分表示は別サービスとして追加しやすいです。

10.2 永続化付き ID allocator

今の ExternalIdAllocator はデモ用の in-memory 実装です。実運用では、

- (system_name, trade_id, revision_no)
- (system_name, component_key, revision_no)

で durable に管理する repository が必要です。

10.3 booking 向け mapper

contract family + representation に応じて booking payload を作る mapper を追加できます。

10.4 cashflow の構造化

今は amount_description を文字列にしていますが、将来的には

- formula AST
- booking-ready field set
- downstream payload fragment

などに置き換えられます。

まとめ

この設計では、

- 契約の正本は product-specific な typed spec
- 取扱いの違いは representation
- 修正履歴は snapshot

- 元の schedule は archive
- 外部システムIDは trade / option leg ごとの linkage

という役割分担にしています。

特に、

- 個別 cashflow 修正
- schedule 全体修正
- representation 切り替え
- trade 単位ID採番
- option leg 単位ID採番

をすべてサポートしつつ、契約本体は経済条件の表現に集中させています。